

Модуль 3. Раздел 1.

1. Войдите под пользователем user1 из практики 2 (su - user1):

```
root@eltex-practice2-v30:~# su - user1
```

2. Подсчитайте количество процессов, имеющих несколько потоков выполнения:

```
user1@eltex-practice2-v30:~$ ps -eo nlwp= | awk '$1 > 1 {count++} END {print count}'
10
user1@eltex-practice2-v30:~$
```

3. Запустите top и настройте вывод полей с информацией о процессе следующим образом:

- удалите поля VIRT, RES, SHR;
- добавьте поле RUSER и сделайте так, чтобы это поле было показано после поля USER;

```
Fields Management for window 1:Def, whose current sort field is RUSER
Navigate with Up/Dn, Right selects for move then <Enter> or Left commits,
'd' or <Space> toggles display, 's' sets sort. Use 'q' or <Esc> to end!

* PID      = Process Id          nsMNT      = MNT namespace Inode
* USER     = Effective User Name nsNET      = NET namespace Inode
* RUSER    = Real User Name      nsPID      = PID namespace Inode
* PR       = Priority            nsUSER     = USER namespace Inode
* NI       = Nice Value          nsUTS      = UTS namespace Inode
* VIRT     = Virtual Image (KiB) LXC         = LXC container name
* RES      = Resident Size (KiB) RSan          = RES Anonymous (KiB)
* SHR      = Shared Memory (KiB) RSfd          = RES File-based (KiB)
* S        = Process Status      RSlk       = RES Locked (KiB)
* %CPU     = CPU Usage            RSsh       = RES Shared (KiB)
* %MEM     = Memory Usage (RES)   CGNAME     = Control Group name
* TIME+    = CPU Time, hundredths NU           = Last Used NUMA node
* COMMAND  = Command Name/Line   LOGID      = Login User Id
```

```
File Edit View SCP Tools Help
top - 09:53:26 up 23 days, 1 min, 2 users, load average: 0.00, 0.00, 0.00
Tasks: 114 total, 1 running, 113 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 s
MiB Mem : 3537.1 total, 1589.8 free, 562.9 used, 1711.4 buff/cache
MiB Swap: 3185.0 total, 3185.0 free, 0.0 used. 2974.3 avail Mem

  PID USER   RUSER   PR  NI  S   %CPU  %MEM    TIME+  COMMAND
  81153 user1   user1   20   0  S    0.0   0.2    0:00.00 bash
  81197 user1   user1   20   0  R    0.0   0.2    0:00.08 top
  59579 systemd+ systemd+ 20   0  S    0.0   0.2    0:01.19 systemd-timesyn
  59577 systemd+ systemd+ 20   0  S    0.0   0.4    0:00.50 systemd-resolve
  59603 systemd+ systemd+ 20   0  S    0.0   0.3    0:08.05 systemd-network
  49398 syslog syslog   20   0  S    0.0   0.2    0:00.48 rsyslogd
```

4. В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля:

```
user1@eltex-practice2-v30:~$ passwd
Changing password for user1.
Current password: _
```

5. Перейдите в терминальное окно с top и выполните следующие действия:

- выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;

```
Mem Swap: 3185.0 total, 3185.0
Which user (blank for all) user1
PID USER RUSER PR NI
```

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
81153	user1	user1	20	0	S	0.0	0.2	0:00.00	bash
81197	user1	user1	20	0	R	0.0	0.2	0:06.23	top
81496	user1	user1	20	0	S	0.0	0.2	0:00.01	bash

- найдите процесс, запущенный командой passwd;

81496	user1	user1	20	0	S	0.0	0.2	0:00.01	bash
81507	root	user1	20	0	S	0.0	0.1	0:00.00	passwd
81516	root	root	20	0	I	0.0	0.0	0:00.00	kworker/0

- отправьте этому процессу сигналы 15 (SIGTERM), 2 (SIGINT), 3 (SIGQUIT), 9(SIGKILL):

15 (SIGTERM)

```
Mem Swap: 3185.0 total, 3185.0 free
Send pid 81507 signal [15/sigterm] 15
```

```
user1@eltex-practice2-v30:~$ passwd
Changing password for user1.
Current password:
```

2 (SIGINT)

```
Mem Swap: 3185.0 total, 3185.0 free
Send pid 81507 signal [15/sigterm] 2
```

PID	USER	RUSER	PR	NI	S	%CPU
-----	------	-------	----	----	---	------

```
user1@eltex-practice2-v30:~$ passwd
Changing password for user1.
Current password:
```

3 (SIGQUIT)

```
Mem Swap: 3185.0 total, 3185.0 free, 0.0 used, 2117.7 av
```

Send pid 81507 signal [15/sigterm] 3

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
81507	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

```
user1@eltex-practice2-v30:~$ passwd
Changing password for user1.
Current password: _
```

9(SIGKILL)

```
Send pid 81507 signal [15/sigterm] 9
```

PID	USER	RUSER	PR	NI	S	%CPU	%MEM	TIME+	COMMAND
81496	user1	user1	20	0	S	0.0	0.2	0:00.01	bash
81507	root	user1	20	0	S	0.0	0.1	0:00.00	passwd

```
user1@eltex-practice2-v30:~$ passwd
Changing password for user1.
Current password: Killed
user1@eltex-practice2-v30:~$
```

6. Выполните команду `vim ~/file_task3.txt` и нажмите `Ctrl-Z`:

```
~/file_task3.txt [New] ^Z 0,0-1 All
[1]+ Stopped vim ~/file_task3.txt
user1@eltex-practice2-v30:~$
```

7. Выполните команду `sleep 600`, нажмите `Ctrl-Z` и выполните команду `jobs`:

```
user1@eltex-practice2-v30:~$ sleep 600
^Z
[2]+ Stopped sleep 600
user1@eltex-practice2-v30:~$ jobs
[1]- Stopped vim ~/file_task3.txt
[2]+ Stopped sleep 600
user1@eltex-practice2-v30:~$
```

8. Последнее задание (`sleep 600`) сделайте фоновым:

```
user1@eltex-practice2-v30:~$ bg %2
[2]+ sleep 600 &
user1@eltex-practice2-v30:~$ _
```

9. Измените число NICE у задания (`sleep 600`), сделав его равным 10:

```
user1@eltex-practice2-v30:~$ jobs -l
[1]+ 81761 Stopped vim ~/file_task3.txt
[2]- 81763 Running sleep 600 &
user1@eltex-practice2-v30:~$ renice 10 -p 81763
81763 (process ID) old priority 0, new priority 10
user1@eltex-practice2-v30:~$ _
```

10. Проверьте, что число NICE у этого задания изменилось:

```
user1@eltex-practice2-v30:~$ ps -o pid,ni,cmd -p 81763
PID  NI  CMD
81763 10  sleep 600
user1@eltex-practice2-v30:~$
```

11. Сделайте задание `vim ~/file_task3.txt` активным и выйдите из редактора:

```
user1@eltex-practice2-v30:~$ fg %1_
```

```
~
:q!_
```

12. Отправьте сигнал 15 (SIGTERM) заданию `sleep 600` и выполните команду `jobs`:

```
user1@eltex-practice2-v30:~$ jobs
[2]+  Running                  sleep 600 &
user1@eltex-practice2-v30:~$ kill -15 %2
user1@eltex-practice2-v30:~$ jobs
[2]+  Terminated              sleep 600
user1@eltex-practice2-v30:~$ _
```

13. Создайте перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (используйте встроенную команду `trap`) и отправьте сигналы самому себе:

```
user1@eltex-practice2-v30:~$ trap 'echo "Меня голыми руками не возьмёшь!"' SIGI
NT SIGQUIT
```

```
user1@eltex-practice2-v30:~$ kill -SIGINT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-v30:~$ kill -SIGQUIT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-v30:~$ _
```

Модуль 3. Раздел 2.

1. Создайте скрипт на языке `bash` с именем `cuckoo.sh`, делающий следующее:

- Создает именованный канал с именем `cuckoo.pid` в каталоге `/tmp/run/`
- Записывает в файл `cuckoo.log` сообщение в формате: ДАТА ВРЕМЯ Startup!
- В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?» возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его

идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N

- При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу.

```
root@eltex-practice2-v30:~# cat cuckoo.sh
#!/bin/bash

PIPE_DIR="/tmp/run"
PIPE="$PIPE_DIR/cuckoo.pid"
LOGFILE="$PWD/cuckoo.log"

mkdir -p "$PIPE_DIR"
rm -f "$PIPE"
mkfifo "$PIPE"

log_msg() {
    echo "$(date '+%F %T') $1" >> "$LOGFILE"
}

shutdown() {
    log_msg "Shutdown!"
    rm -f "$PIPE"
    exit 0
}

trap shutdown SIGTERM

log_msg "Startup!"

exec 3<>"$PIPE"

while true; do
    if read -r line <&3; then
        if [[ "$line" =~ ^([A-Za-z0-9._-]+\)[\([0-9]+\)\]:[[:space:]]how[[:space:]]much[[:space:]]time[[:space:]]do[[:space:]]I[[:space:]]have\?$ ]]; then
            name="${BASH_REMATCH[1]}"
            pid="${BASH_REMATCH[2]}"
            n=$((RANDOM % 9 + 2))
            log_msg "$name[$pid] $n"
        fi
    fi
done

root@eltex-practice2-v30:~# _
```

2. Запустите скрипт cuckoo.sh при помощи nohup или screen:

```
root@eltex-practice2-v30:~# nohup ./cuckoo.sh > /dev/null 2>&1 &
[1] 1494
root@eltex-practice2-v30:~# pgrep -af cuckoo.sh
1494 /bin/bash ./cuckoo.sh
root@eltex-practice2-v30:~# ls -l /tmp/run/cuckoo.pid
prw-r--r-- 1 root root 0 Aug 12 14:20 /tmp/run/cuckoo.pid
root@eltex-practice2-v30:~# cat cuckoo.log
2026-08-12 14:20:30 Startup!
```

3. Создайте скрипт на языке bash с именем `template_task.sh`, делающий следующее:

- При запуске проверяет, что имя скрипта не совпадает с `template_task.sh`, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
- При запуске дописывает в файл `report_имя_скрипта_без_полного_пути.log` в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
- Запрашивает через именованный канал скрипта `cuckoo` время сообщением в указанном ранее формате и ждет такое количество секунд.
- Дописывает в файл `report_имя_скрипта_без_полного_пути.log` сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.

```
root@eltex-practice2-v30:~# cat template_task.sh
#!/bin/bash

SCRIPT_NAME="$(basename "$0")"
PID="$$_"
REPORT_FILE="$PWD/report_${SCRIPT_NAME}.log"
CUCKOO_PIPE="/tmp/run/cuckoo.pid"
CUCKOO_LOG="$PWD/cuckoo.log"

log_msg() {
    echo "$(date '+%F %T') [$PID] $1" >> "$REPORT_FILE"
}

if [[ "$SCRIPT_NAME" == "template_task.sh" ]]; then
    echo "я бригадир, сам не работаю"
    exit 0
fi

log_msg "Скрипт запущен"

REQUEST="${SCRIPT_NAME}[$PID]: how much time do I have?"
echo "$REQUEST" > "$CUCKOO_PIPE"

N=""
while true; do
    N=$(awk -v target="${SCRIPT_NAME}[$PID]" '$3 == target {print $4}' "$CUCKOO_LOG" | tail -n 1)
    if [[ "$N" =~ ^[2-9]$|^10$ ]]; then
        break
    fi
    sleep 1
done

sleep "$N"

log_msg "Скрипт завершился, работал $N секунд."
root@eltex-practice2-v30:~# _
```

```
root@eltex-practice2-v30:~# chmod +x template_task.sh
root@eltex-practice2-v30:~# ./template_task.sh
я бригадир, сам не работаю
root@eltex-practice2-v30:~# cp template_task.sh worker1.sh
root@eltex-practice2-v30:~# chmod +x worker1.sh
root@eltex-practice2-v30:~# _
```

```
root@eltex-practice2-v30:~# ./worker1.sh
root@eltex-practice2-v30:~# cat cuckoo.log
2026-08-12 14:20:30 Startup!
2026-08-12 14:28:12 worker1.sh[1642] 5
root@eltex-practice2-v30:~# cat report_worker1.sh.log
2026-08-12 14:28:12 [1642] Скрипт запущен
2026-08-12 14:28:17 [1642] Скрипт завершился, работал 5 секунд.
```

4. Создайте скрипт на языке bash с именем `observer.sh`, читающий файл конфигурации со списком скриптов `observer.conf`, проверяющим их наличие в списке работающих процессов поиском в `/proc` и запускающих их в отключенном от терминала режиме (`nohup`) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл `observer.log`


```

root@eltex-practice2-v30:~# cat observer.sh
#!/bin/bash

CONF_FILE="$PWD/observer.conf"
LOG_FILE="$PWD/observer.log"

log_msg() {
    echo "$(date '+%F %T') $1" >> "$LOG_FILE"
}

is_running() {
    local script_name="$1"

    for pid_dir in /proc/[0-9]*; do
        [[ -r "$pid_dir/cmdline" ]] || continue

        cmdline=$(tr '\0' ' ' < "$pid_dir/cmdline" 2>/dev/null)

        if [[ "$cmdline" == *"$script_name"* ]]; then
            return 0
        fi
    done

    return 1
}

while IFS= read -r script || [[ -n "$script" ]]; do
    [[ -z "$script" ]] && continue

    if ! is_running "$script"; then
        nohup "$script" > /dev/null 2>&1 &
        log_msg "Перезапуск $script"
    fi
done < "$CONF_FILE"
root@eltex-practice2-v30:~#

```

```

root@eltex-practice2-v30:~# cat observer.conf
cuckoo.sh
worker1.sh
root@eltex-practice2-v30:~# _

```

```

root@eltex-practice2-v30:~# pgrep -af worker1.sh
root@eltex-practice2-v30:~# ./observer.sh
root@eltex-practice2-v30:~# cat observer.log
2026-08-12 14:40:04 Перезапуск worker1.sh
root@eltex-practice2-v30:~# cat report_worker1.sh.log
2026-08-12 14:28:12 [1642] Скрипт запущен
2026-08-12 14:28:17 [1642] Скрипт завершился, работал 5 секунд.
2026-08-12 14:40:04 [1981] Скрипт запущен
2026-08-12 14:40:12 [1981] Скрипт завершился, работал 8 секунд.
root@eltex-practice2-v30:~# cat cuckoo.log
2026-08-12 14:20:30 Startup!
2026-08-12 14:28:12 worker1.sh[1642] 5
2026-08-12 14:40:04 worker1.sh[1981] 8
root@eltex-practice2-v30:~# _

```


5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту

```
root@eltex-practice2-v30:~# crontab -e
no crontab for root - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1_
```

```
# m h dom mon dow    command
* * * * * cd /root && /root/observer.sh_
```

```
crontab: installing new crontab
root@eltex-practice2-v30:~# cat report_worker1.sh.log
2026-08-12 14:28:12 [1642] Скрипт запущен
2026-08-12 14:28:17 [1642] Скрипт завершился, работал 5 секунд.
2026-08-12 14:40:04 [1981] Скрипт запущен
2026-08-12 14:40:12 [1981] Скрипт завершился, работал 8 секунд.
2026-08-12 14:46:01 [2301] Скрипт запущен
2026-08-12 14:46:06 [2301] Скрипт завершился, работал 5 секунд.
2026-08-12 14:47:01 [2571] Скрипт запущен
2026-08-12 14:47:03 [2571] Скрипт завершился, работал 2 секунд.
root@eltex-practice2-v30:~# cat observer.log
2026-08-12 14:40:04 Перезапуск worker1.sh
2026-08-12 14:46:01 Перезапуск worker1.sh
2026-08-12 14:47:01 Перезапуск worker1.sh
root@eltex-practice2-v30:~# cat cuckoo.log
2026-08-12 14:20:30 Startup!
2026-08-12 14:28:12 worker1.sh[1642] 5
2026-08-12 14:40:04 worker1.sh[1981] 8
2026-08-12 14:46:01 worker1.sh[2301] 5
2026-08-12 14:47:01 worker1.sh[2571] 2
root@eltex-practice2-v30:~#
```

6. Создайте несколько символических ссылок на файл template_task.sh с различными именами (рабочие задачи), добавьте в файл конфигурации observer.conf соответствующие записи об этих задачах, включая исходный файл template_task.sh

```
root@eltex-practice2-v30:~# ln -s template_task.sh worker2.sh
root@eltex-practice2-v30:~# ln -s template_task.sh worker3.sh
root@eltex-practice2-v30:~# ln -s template_task.sh worker4.sh
```

```

root@eltex-practice2-v30:~# ls -l worker*.sh template_task.sh
-rwxr-xr-x 1 root root 748 Aug 12 14:25 template_task.sh
-rwxr-xr-x 1 root root 748 Aug 12 14:26 worker1.sh
lrwxrwxrwx 1 root root 16 Aug 13 10:16 worker2.sh -> template_task.sh
lrwxrwxrwx 1 root root 16 Aug 13 10:16 worker3.sh -> template_task.sh
lrwxrwxrwx 1 root root 16 Aug 13 10:16 worker4.sh -> template_task.sh
root@eltex-practice2-v30:~# _

```

```

root@eltex-practice2-v30:~# cat observer.conf
cuckoo.sh
worker1.sh
template_task.sh
worker2.sh
worker3.sh
worker4.sh
root@eltex-practice2-v30:~# _

```

7. Соберите статистику работы в виде набора файлов report_*.log, observer.log, cuckoo.log приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива tar. Не забудьте остановить процесс, удалив задачу в cron!

```

root@eltex-practice2-v30:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
root@eltex-practice2-v30:~# _

```

```
root@eltex-practice2-v30:~# pkill -f observer.sh
root@eltex-practice2-v30:~# pkill -f cuckoo.sh
root@eltex-practice2-v30:~# pkill -f worker1.sh
[1]+  Done                  nohup ./cuckoo.sh > /dev/null 2>&1
root@eltex-practice2-v30:~# pkill -f worker2.sh
root@eltex-practice2-v30:~# pkill -f worker3.sh
root@eltex-practice2-v30:~# pkill -f worker4.sh
root@eltex-practice2-v30:~# pkill -f template_task.sh
root@eltex-practice2-v30:~# pgrep -af observer.sh
root@eltex-practice2-v30:~# pgrep -af cuckoo.sh
```

```
root@eltex-practice2-v30:~# ls -l *.sh observer.conf report_*.log observer.log
cuckoo.log
cuckoo.log
cuckoo.sh
observer.conf
observer.log
observer.sh
report_worker1.sh.log
report_worker2.sh.log
report_worker3.sh.log
report_worker4.sh.log
template_task.sh
worker1.sh
worker2.sh
worker3.sh
worker4.sh
root@eltex-practice2-v30:~#
```

```
root@eltex-practice2-v30:~# tar -tzvf task3_report.tar.gz
-rwxr-xr-x root/root      630 2026-08-12 14:15 cuckoo.sh
-rwxr-xr-x root/root      748 2026-08-12 14:25 template_task.sh
-rwxr-xr-x root/root      614 2026-08-12 14:33 observer.sh
-rw-r--r-- root/root       71 2026-08-12 14:49 observer.conf
-rwxr-xr-x root/root      748 2026-08-12 14:26 worker1.sh
lrwxrwxrwx root/root       0 2026-08-13 10:16 worker2.sh -> template_task.sh
lrwxrwxrwx root/root       0 2026-08-13 10:16 worker3.sh -> template_task.sh
lrwxrwxrwx root/root       0 2026-08-13 10:16 worker4.sh -> template_task.sh
-rw-r--r-- root/root 179476 2026-08-13 10:20 report_worker1.sh.log
-rw-r--r-- root/root   616 2026-08-13 10:20 report_worker2.sh.log
-rw-r--r-- root/root   616 2026-08-13 10:20 report_worker3.sh.log
-rw-r--r-- root/root   617 2026-08-13 10:20 report_worker4.sh.log
-rw-r--r-- root/root 311746 2026-08-13 10:20 observer.log
-rw-r--r-- root/root  49171 2026-08-13 10:23 cuckoo.log
root@eltex-practice2-v30:~# ls
```